

Robot Movement Planning

A Constraint Programming approach to gripper-aware depalletizing

Alessio Russo

1 Problem Statement

We consider a rectangular layer of boxes arranged on a discrete grid. Each box has dimensions $x \times y$, where the x -axis is aligned with the belt direction and the y -axis is orthogonal to it. The target packaging layer contains n boxes, whose destination positions are denoted by $(P_x[i], P_y[i])$, for $i = 1, \dots, n$. These positions are assumed to be non-overlapping and to tile a rectangular grid.

A robotic gripper with two parallel plates, each of size S , can pick a contiguous sequence of boxes along the x -axis. The total physical length of the sequence must be at most $S + x$, which corresponds to a maximum number of boxes equal to

$$\left\lfloor \frac{S + x}{x} \right\rfloor.$$

This formulation accounts for a maximum admissible overflow of $x/2$ on each side of the gripper. Figure 1 shows an example of the picking operation.



Figure 1: Example of a picking operation from the input stream.

At each release operation, only one gripper plate can open. Therefore, the opening side must face a free direction, either the external boundary of the layer or a grid position that has not yet been filled. Figure 2 shows an example of this release constraint. This constraint couples the geometric construction of groups with the temporal order in which they are released.

The problem can therefore be decomposed into two related subproblems:

- (A) **Grouping.** Partition the n boxes into groups that are contiguous along the x -axis, belong to the same row, and fit within the gripper capacity.
- (B) **Scheduling.** Determine an ordering of the groups and assign an opening plate side to each group, so that the opening plate always faces a free direction at the time of release.

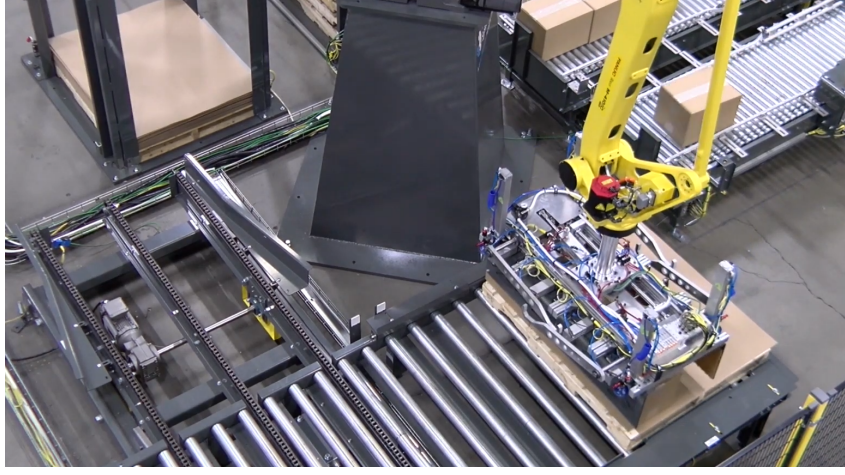


Figure 2: Example of a release operation. Since only one gripper plate can open at a time, the opening side must face either the boundary of the layer or a position that has not yet been filled.

1.1 Parameters and Derived Constants

Based on this formulation, the MiniZinc model uses the following input parameters:

- the total number of boxes in the layer, denoted by `n_boxes`;
- the grid dimensions of the layer, denoted by `boxes_along_x` and `boxes_along_y`;
- the box dimension parallel to the gripper, denoted by `boxes_x_dim`. In the natural orientation, `boxes_x_dim` corresponds to x ; if the layer is rotated, it corresponds to y ;
- the gripper size S , denoted by `grip_size`.

These parameters are declared in MiniZinc as follows:

```

1 par int: n_boxes;
2 par int: boxes_x_dim;
3 par int: boxes_along_x;
4 par int: boxes_along_y;
5 par int: grip_size;

```

From these inputs, the model derives three constants. First, the maximum number of boxes that can be picked in a single operation is computed directly from the gripper geometry:

$$\text{max_pkble_boxes} = \left\lfloor \frac{\text{grip_size} + \text{boxes_x_dim}}{\text{boxes_x_dim}} \right\rfloor.$$

Second, the minimum number of groups required to cover one row is obtained by ceiling division:

$$\text{groups_per_row} = \left\lceil \frac{\text{boxes_along_x}}{\text{max_pkble_boxes}} \right\rceil.$$

Finally, the total number of groups is obtained by multiplying the number of groups per row by the number of rows:

$$\text{n_groups} = \text{boxes_along_y} \cdot \text{groups_per_row}.$$

The corresponding MiniZinc declarations are:

```

1 par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;
2
3 par int: groups_per_row = (boxes_along_x + max_pkble_boxes - 1)
4                      div max_pkble_boxes;
5
6 par int: n_groups = boxes_along_y * groups_per_row;

```

2 Grouping

This part of the model admits several possible CP encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each group explicitly stores the identifiers of all boxes it contains. Since groups may have different lengths, this representation usually requires a two-dimensional array with padding values for shorter groups.

```

1 array[1..n_groups, 1..max_pkble_packs] of var 0..n_packs: groups;

```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes group membership explicit and easy to inspect. However, it introduces a large number of decision variables and requires additional constraints to enforce the structure of a valid solution. In particular, the model must ensure that each box appears exactly once, that non-empty positions inside each group are ordered, and that the groups are consistent with the geometric ordering of the layer.

```

1 % Each pack identifier 1..n_packs appears exactly once.
2 constraint
3   forall(v in 1..n_packs) (
4     count(array1d(groups), v) = 1
5   );
6
7 % Non-empty entries are placed before padding values.
8 constraint
9   forall(g in 1..n_groups, j in 1..max_pkble_packs-1) (
10     groups[g,j] = 0 -> groups[g,j+1] = 0
11   );
12
13 % Pack identifiers are strictly increasing inside each group.
14 constraint
15   forall(g in 1..n_groups, j in 1..max_pkble_packs-1) (
16     groups[g,j] != 0 /\ groups[g,j+1] != 0 ->
17       groups[g,j] < groups[g,j+1]
18   );

```

A second possibility is a *binary assignment* encoding, where a Boolean variable states whether pack i belongs to group g . This representation can be written as follows:

```

1 array[1..n_groups, 1..n_packs] of var bool: belongs_to_group;

```

In this case, group membership is represented by the truth value of `belongs_to_group[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation.

It also does not encode contiguity or same-row membership directly. These properties must be reconstructed through additional channeling constraints.

For this reason, the adopted model represents each group using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each group is described by only two main variables. Contiguity is implicit: once the start index and the length are known, the boxes in the group are uniquely determined. The same-row constraint can also be expressed directly by requiring the interval not to cross a row boundary. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid groups.

```
1 array[1..n_groups] of var 1..n_packs: group_start;
2 array[1..n_groups] of var 1..max_pkble_packs: group_len;
```

The adopted encoding uses two arrays: `group_start`, which stores the first box of each group, and `group_len`, which stores the number of boxes in that group.

```
1 constraint group_start[1] = 1;
2
3 constraint forall(g in 1..n_groups - 1)(
4   group_start[g + 1] = group_start[g] + group_len[g]
5 );
6
7 constraint group_start[n_groups] + group_len[n_groups] - 1 = n_packs;
8
9 constraint forall(g in 1..n_groups)(
10  (group_start[g] - 1) div packs_along_x =
11  (group_start[g] + group_len[g] - 2) div packs_along_x
12 );
```

The first three constraints define the groups as consecutive intervals over the row-major ordering of the packs. The first group starts from pack 1, and each following group starts immediately after the last pack of the previous group. The final constraint in this block forces the last group to end at pack n . Together, these conditions ensure that all packs are covered exactly once, without gaps or overlaps.

The fourth constraint enforces the geometric feasibility of each group. Since the gripper can only pick boxes aligned along the x -axis, a group cannot cross from one row to the next. The constraint therefore compares the row index of the first pack in the group with the row index of the last pack. If the two indices are equal, the whole interval lies on the same row.

3 Scheduling

Once the grouping stage has fixed the horizontal intervals, the second part of the model decides when each group is released and which gripper plate opens. As in the grouping stage, several encodings are possible. A direct time-expanded formulation would introduce a Boolean variable for every pair of group and time step, stating whether a group has already been placed at a given moment.

```
1 array[1..n_groups, 1..n_groups] of var bool: already_placed;
2 array[1..n_groups] of var 0..1: plate;
```

This representation makes the state of the pallet explicit at every step. However, it is unnecessarily large for this problem. It also requires additional channeling constraints to ensure that each group appears at exactly one time step and that the Boolean occupancy matrix is consistent with the chosen order.

The adopted model instead represents the schedule by assigning a single integer position to each group. The array `drop_order` stores the time step at which group g is released, while `plate` stores the opening side used for that release. The value 0 denotes the left plate and the value 1 denotes the right plate.

```
1 array[1..n_groups] of var 1..n_groups: drop_order;
2 array[1..n_groups] of var 0..1: plate;
```

The only global constraint needed to make `drop_order` a valid schedule is `alldifferent`. Since every value belongs to the domain $1, \dots, n_groups$, enforcing pairwise difference means that each release step is used exactly once.

```
1 constraint alldifferent(drop_order);
```

The release feasibility constraint depends on the horizontal neighbors of each group. These neighbors do not need to be decision variables. The grouping constraints force every row to contain exactly `groups_per_row` groups, numbered consecutively in row-major order. Therefore, the position of group g inside its row is given by $(g - 1) \bmod \text{groups_per_row}$, and its left and right neighbors can be computed directly from the group index. The sentinel value 0 is used when a group lies on the boundary of a row.

```
1 array[1..n_groups] of par int: left_neighbor =
2   [ if (g - 1) mod groups_per_row > 0
3     then g - 1
4     else 0
5     endif
6   | g in 1..n_groups ];
7
8 array[1..n_groups] of par int: right_neighbor =
9   [ if (g - 1) mod groups_per_row < groups_per_row - 1
10    then g + 1
11    else 0
12    endif
13   | g in 1..n_groups ];
```

Declaring these arrays as `par` rather than `var` keeps the neighbor structure outside the search space. The solver therefore searches only over the ordering of the groups and the plate side assigned to each release.

The final constraint encodes the physical restriction that, when a group is released, the opening plate must face a free direction. A direction is free if the corresponding neighbor does not exist, meaning the group is on the external boundary of the row, or if that neighbor is scheduled later and is therefore still empty at the current release step.

```
1 constraint forall(g in 1..n_groups)(
2   (plate[g] = 0 /\
3     (left_neighbor[g] = 0 /\
4       drop_order[left_neighbor[g]] > drop_order[g]))
```

```

5  \/  

6  (plate[g] = 1 /\  

7    (right_neighbor[g] = 0 /\  

8      drop_order[right_neighbor[g]] > drop_order[g]))  

9 );

```

For each group g , the first branch of the disjunction corresponds to opening the left plate. This is feasible only if there is no left neighbor, or if the left neighbor has a larger `drop_order` value than g . The second branch applies the same reasoning to the right plate. In this way, the constraint couples the two scheduling decisions: once the plate side is fixed, the model also fixes which neighboring group must be absent at the time of release.

The model is a pure feasibility problem:

```

1  solve satisfy;

```

No objective function is required, since any assignment of `drop_order` and `plate` that satisfies the neighbor constraint is a valid pick-and-place plan.

4 Driving the model from Python

The Python layer is intentionally thin — it is a wrapper around `minizinc-python` that handles the geometry computations the model deliberately leaves out, and produces visual artifacts of solutions. Three scripts compose the pipeline:

sample.py Solves a single instance and renders the layout and a step-by-step drop schedule as PDFs. The Python side is responsible for picking the orientation that fits more boxes on the pallet (`lib/utils.py: gen_pallet_info`), deriving `boxes_x_dim` and the gripper size $S = 3x$, and translating the model output (the four arrays `group_start`, `group_len`, `drop_order`, `plate`) into a high-level *schedule* of drop actions.

benchmark.py Runs the same model across the cartesian product of five pallet presets and six pack sizes, against every available solver, with a 5-minute timeout. Each run is encapsulated and timing/error information is recorded in a single `benchmark/results.json` file.

plot_benchmark.py Consumes the JSON benchmark and produces the comparison plots referenced in Section 5.

The renderer in `lib/visualize.py` is orientation-agnostic: it consumes already-rotated cell dimensions (`cell_w`, `cell_h`) and the four solution arrays, and outputs both an overall pallet figure (colored by group) and one figure per drop step (with an arrow showing the opening plate side).

4.1 Worked example

To make the output of the model concrete, we run `sample.py` on a 70×70 pack and an 800×1200 pallet. The geometry helper chooses the rotation that maximises pack count, yielding an 11×17 grid ($n = 187$ boxes). With $S = 3 \cdot 70 = 210$, the gripper capacity is $\lfloor (210 + 70)/70 \rfloor = 4$ boxes, and each row of 11 boxes is split into 3 groups of sizes $3 + 4 + 4$, for a total of $n_groups = 51$.

Figure 4 samples four steps from the 51-step schedule produced by the model. The opening plate is highlighted by a red border on the active group and a blue arrow pointing in the direction the plate opens; the side it points to is always empty at the moment of release.



Figure 3: Pallet layout produced by the grouping stage: 51 groups tiling the 11×17 grid in row-major order, each colored distinctly. Groups alternate 3–4–4 across each row.

The schedule starts in the bottom-right corner and works back toward the top-left: at step 1 the gripper drops G_{51} opening to the left (its left neighbor G_{50} has not yet been placed), and the final step places G_1 opening to the left into the pallet edge (no left neighbor exists, so the boundary sentinel applies).

5 Benchmarking

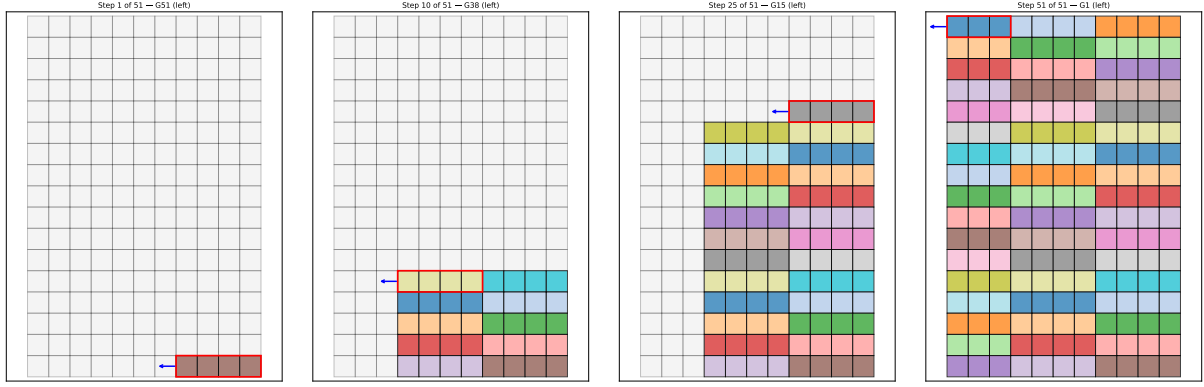
5.1 Setup

The benchmark runs the MiniZinc model on every (pallet, pack) combination from the lists in `benchmark.py` (see Table 1), against every solver registered with the local MiniZinc installation, with a per-run timeout of 300 seconds.

The gripper size is fixed at $S = 3 \cdot \text{boxes_x_dim}$, the maximum allowed by the project specification.

5.2 Overall scaling

Figure 5 shows solve time as a function of the total number of boxes on the pallet, on a logarithmic axis, for every solver across all instances. The geometric mean and min/max range per solver is



(a) Step 1: G_{51} , bottom-right corner. (b) Step 10: G_{38} opens left into a still-empty cell. (c) Step 25: G_{15} , about halfway through. (d) Step 51: G_1 , the final drop, opens left into the pallet boundary.

Figure 4: Selected drop steps from the 51-step schedule. The red border marks the group being placed; the blue arrow shows the opening plate side, which always faces a free cell.

Pallet preset	Dimensions (mm)	Pack size (mm)
Europallet	1200×800	50×50
Industrie	1200×1000	70×70
US 40×48	1219×1016	100×100
Half-pallet	800×600	120×80
Australian	1165×1165	150×150
		200×200

Table 1: Pallet presets and pack sizes used in the benchmark.

summarised in Figure 6.

5.3 Per-pallet breakdown

Splitting the data by pallet preset (Figure 7) isolates the effect of pack count from pallet shape: for a fixed pallet, the curves are functions of pack count alone, while differences between pallets reflect the impact of `boxes_along_x` and `boxes_along_y` on the number of groups.

5.4 Wall time vs. solve time

The gap between Python-measured wall time and solver-reported solve time captures MiniZinc flattening and inter-process overhead. Figure 8 plots the two against each other; points above the diagonal indicate runs in which the harness cost dominated the search itself.

6 Conclusion

The two-stage formulation — grouping the pallet into pickable runs, then scheduling the drops with a plate side per group — maps cleanly onto MiniZinc decision variables and yields a compact, fully feasible model. The key modelling choice is to fix the number of groups statically and force the partition into a prefix-sum shape; this lets us pre-compute the neighbor structure as parameters rather than variables, keeping the plate-collision constraint a simple disjunction over indexed lookups.

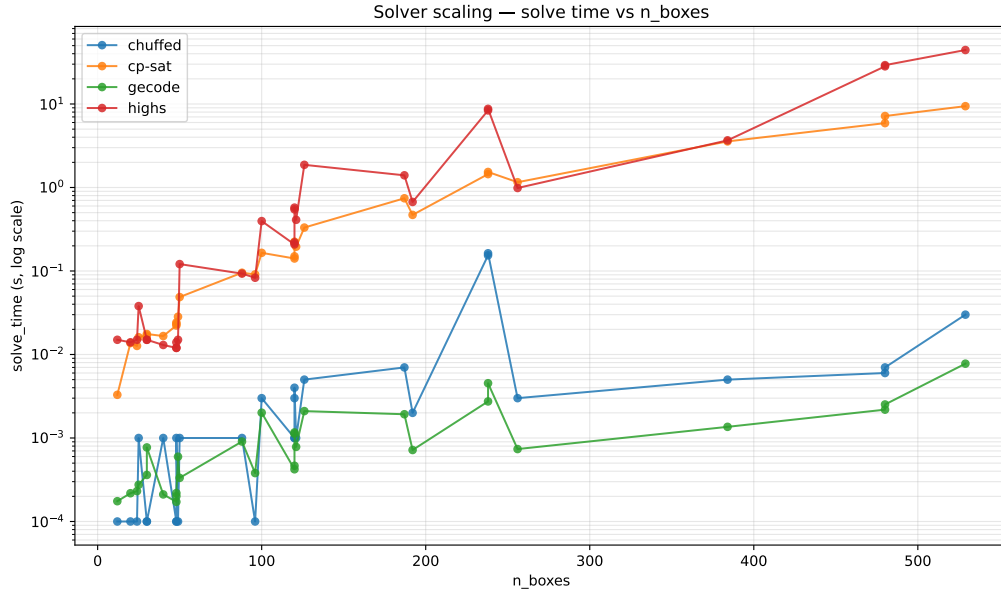


Figure 5: Solve time vs. total number of boxes — all solvers, all instances. Log scale on the time axis.

The benchmark confirms that the model scales well across realistic pallet sizes within the 5-minute budget, and exposes the expected per-solver trade-offs between flattening overhead and search performance.

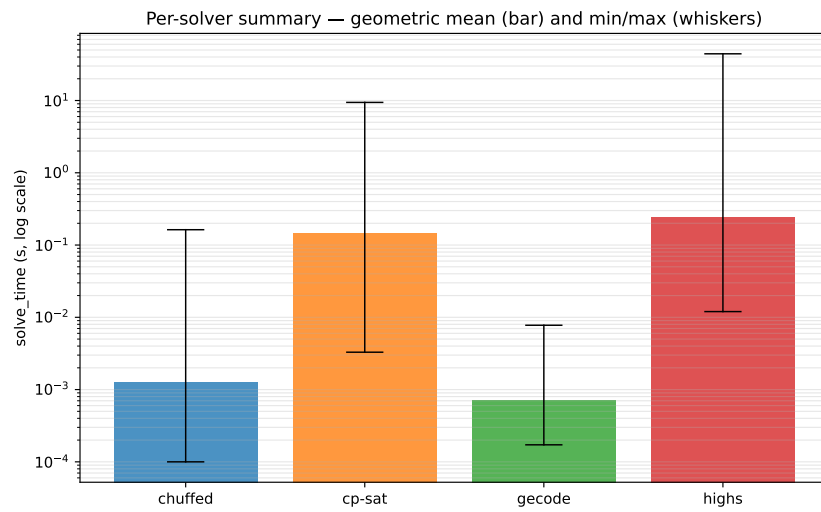
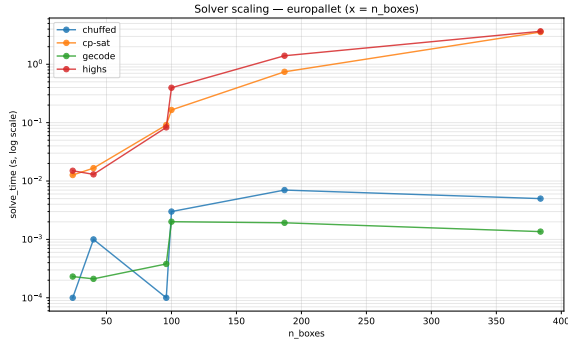
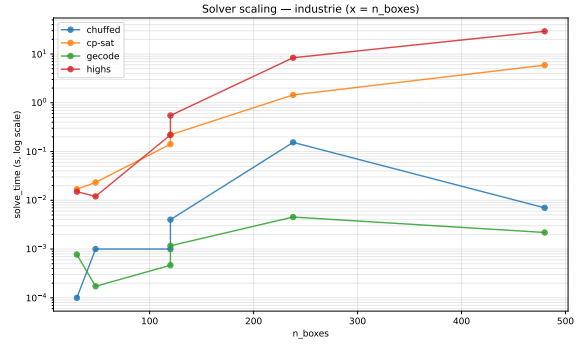


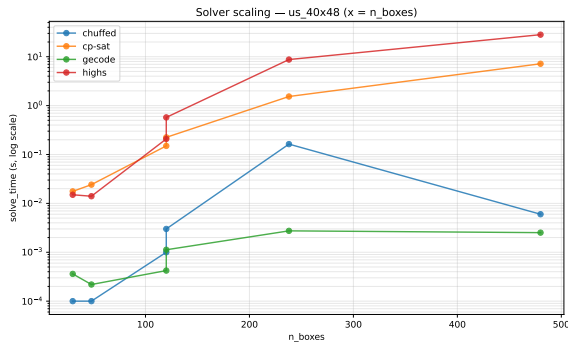
Figure 6: Per-solver geometric mean of solve time (bar) with min/max whiskers across the full benchmark set.



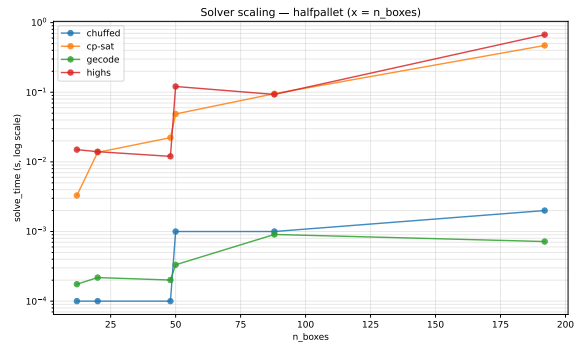
(a) Europallet



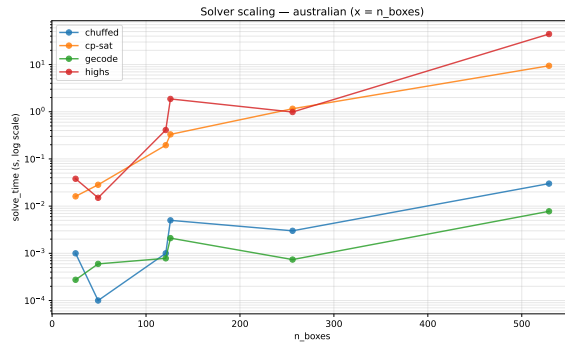
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 7: Solve time vs. number of boxes, broken down by pallet preset.

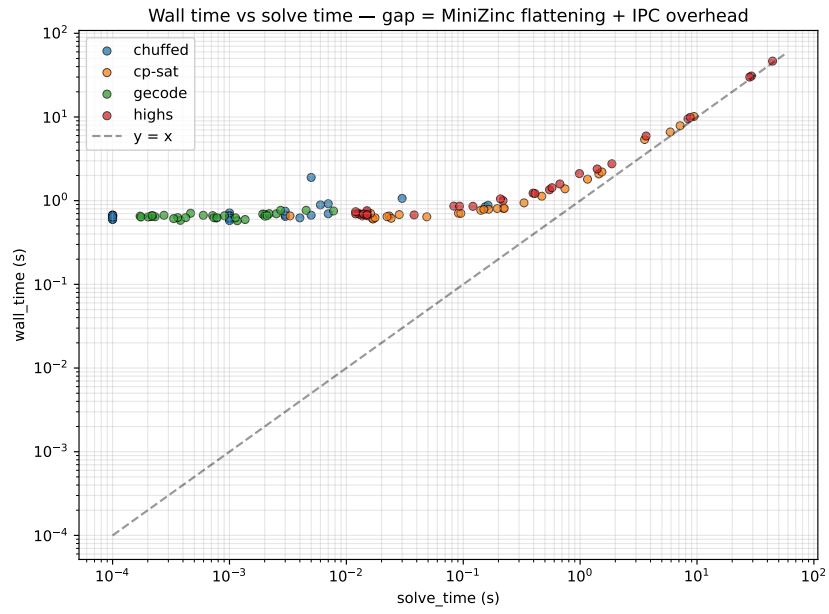


Figure 8: Wall time vs. solve time per run, log-log. The diagonal represents zero overhead; all points lie above it.